Node.js Development inside the ict-api Container

Using the **ict-api** service container as your Node.js development environment, driven entirely from VS Code on Windows.

Assumptions

Item	Setup
OS	Windows 11
IDE	Visual Studio Code, installed on Windows
Extension	**Dev Containers** (ms-vscode-remote.remote-containers) installed in VS Code
Containers	WSL 2 + Docker Desktop (WSL 2 backend)
	Container 'ict-api' is running
Project location	nap-dev-fullstack lives inside the WSL filesystem, e.g. `\\wsl.localhost\Ubuntu\home\<username>\Projects\containers\nap-dev-fullstack`

> **Before you start:** the stack must be running so that **ict-api** exists as a live container. From a terminal in the project folder (WSL or Windows):

```
docker compose up -d
docker container ls   # ict-api should show "Up ... (healthy)"
```

1. Navigate to the project folder and start VS Code

1. Open **Microsoft File Explorer** (press **Win + E**).
2. Click in the **address bar** at the top, paste the WSL path to the project folder, and press **Enter**:
3. You should see `docker-compose.yml`, `README.md`, and the `workspace` folder.
4. Start VS Code on this folder: **right-click in empty space** inside the folder and choose **"Open with Code"** (or right-click the folder itself from the parent directory).
5. VS Code opens with the project folder loaded. On first open, answer **"Trust"** when asked whether you trust the authors of the files in this folder.

```
\\wsl.localhost\Ubuntu\home\<username>\Projects\containers\nap-dev-fullstack
```

Replace `Ubuntu` with your WSL distribution name and `<username>` with your Linux user. Alternatively, use the **Linux penguin icon** in the Explorer navigation pane: `Linux > <Distro> > home > <username> > ...`

> If the "Open with Code" entry is missing, re-run the VS Code installer and enable "Add 'Open with Code' action to the Windows Explorer context menu", or open VS Code separately and use **File > Open Folder**.

You are now in the **host window** - it edits `docker-compose.yml`, `README.md`, and everything under `workspace/` directly on the WSL filesystem.

2. Attach VS Code to the running ict-api container

1. With the project open in VS Code, press **Ctrl + Shift + P** to open the **Command Palette**.

2. Type and select:
3. A container picker drops down at the top of the window. Select the entry for **ict-api** (image `node:26-trixie`).
4. A **new VS Code window** opens, connected to the container. The **green remote indicator** in the bottom-left corner shows `Container ict-api`. This window's editor, terminal, and file system all operate **inside the container**.

```
Dev Containers: Attach to Running Container...
```

> **Alternative route:** install the **Docker** extension, find **ict-api** in the Docker view's *Containers* list, right-click it, and choose **Attach Visual Studio Code** - same result.

Notes for this scaffold:

- No `devcontainer.json` is needed; **Attach to Running Container** works against any running container.
- On the first attach, VS Code installs its server component into the container at `/home/node/.vscode-server`. This lives in the container's writable layer, so it is re-downloaded if the container is recreated (`docker compose down` + `up`).
- Do **not** attach to **ict-db** - the SQL Server image is not a development environment.

3. Navigate to /workspace inside the container window

The attached window opens without a folder. Load the bind-mounted workspace:

1. In the **ict-api** window, choose **File > Open Folder...** (or press **Ctrl + K** then **Ctrl + O**).
2. In the folder picker's path box, type `/workspace` and confirm. The window reloads with `/workspace` as the project root.
3. Verify where you are - open the integrated terminal (press **Ctrl +** `) and run:

```
pwd          # -> /workspace
node -v      # -> v26.x  (the Node.js runtime you develop against)
npm -v
```

What /workspace means here:

- It is the bind mount of the host folder `workspace/api`. Files you create or edit in this window appear under `workspace/api` on your WSL filesystem **immediately** - no copy, no rebuild.
- Edits made in the host window (step 1) appear in this container window just as fast, in both directions.
- The repo root is **not** mounted into the container: `docker-compose.yml`, `README.md`, and the sibling `workspace/web` / `workspace/db` folders are not visible from `/workspace`. Edit those in the host window.

Start your Node.js project exactly as you would anywhere else:

```
npm init -y
npm install express   # add whatever your API needs
```

Everything, including `node_modules`, lands on the bind mount and survives container restarts.

Finishing up

To leave the container without stopping it:

- Click the green `Container ict-api` indicator (bottom-left) > **Close Remote Connection**, or
- **Ctrl + Shift + P** > **Remote: Close Remote Connection**.

The container keeps running; your files stay on the host under `workspace/api`. Re-attach any time with **Dev Containers: Attach to Running Container...**.

Quick reference

Step	Action
Open project	File Explorer > \\wsl.localhost\<Distro>\home\<user>\...\nap-dev-fullstack > right-click > **Open with Code**
Attach	**Ctrl+Shift+P** > **Dev Containers: Attach to Running Container...** > pick **ict-api**
Load workspace	In the attached window: **File > Open Folder** > `/workspace`
Verify	Terminal: `pwd` > `/workspace`, `node -v`
Leave	Remote indicator > **Close Remote Connection** (container keeps running)